

```

1 #=====
2 # MIT 6.046J 代码精读 • PS2 - 通配符匹配 (FFT 卷积)
3 # 源文件: wildcard_match.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS2 Problem 2-1: wildcard pattern matching via polynomial multiplication.
7
8 Given source S over {a,b} and pattern P over {a,b,*} with * a single-symbol
9 wildcard, output all j such that P matches S[j..j+m-1].
10
11 Reduction (part b), binary alphabet {a,b}:
12 Map a -> +1, b -> -1 in the *source*. For the *pattern*, at a non-wildcard
13 position use the same +1 code, and at a wildcard use 0 (so it never
14 contributes a mismatch). Reverse the pattern. For a placement j, the
15 correlation
16     c_j = sum_{k : P[k] != *} s(S[j+k]) * s(P[k])
17 equals (number of fixed positions) exactly when every fixed symbol matches
18 (each matched fixed pair contributes +1: (+1)(+1) or (-1)(-1)), and is
19 strictly less otherwise. A correlation is a convolution of S with reversed P,
20 so one FFT polynomial multiplication yields all c_j at once.
21
22 Part (c): with FFT the total time is O(n log n) (m <= n).
23
24 Part (d), larger alphabet {A,C,G,T}: a single +1 code no longer distinguishes
25 symbols. Use the standard "sum of per-symbol indicator convolutions" trick:
26 for each alphabet symbol sigma, indicator_S[i]=1 iff S[i]=sigma, and
27 indicator_P[k]=1 iff P[k]=sigma (0 at wildcards for all sigma). The number of
28 matched fixed positions at placement j is sum_sigma (indicatorS_sigma *
29 reversed indicatorP_sigma)[...]; it equals the number of non-wildcard pattern
30 positions exactly at a full match. |alphabet| FFT multiplications => still
31 O(n log n) for constant alphabet.
32 """
33 from __future__ import annotations
34
35 from typing import List
36
37 from fft import multiply
38
39 BIN = {"a": 1, "b": -1}
40
41
42 def match_binary(S: str, P: str) -> List[int]:
43     """Wildcard matching over {a,b} using ONE FFT multiplication (parts b,c)."""
44     n, m = len(S), len(P)
45     if m > n:
46         return []
47     fixed = sum(1 for ch in P if ch != "*")
48     s = [BIN[ch] for ch in S]
49     # reversed pattern code: wildcard -> 0
50     pr = [0 if ch == "*" else BIN[ch] for ch in reversed(P)]
51     conv = multiply(s, pr)
52     # correlation for placement j lives at index j + (m-1)
53     out = []
54     for j in range(n - m + 1):
55         if conv[j + m - 1] == fixed:
56             out.append(j)
57     return out
58
59
60 def match_alphabet(S: str, P: str, alphabet: str = "ACGT") -> List[int]:
61     """Wildcard matching over an arbitrary constant alphabet (part d)."""
62     n, m = len(S), len(P)
63     if m > n:
64         return []
65     fixed = sum(1 for ch in P if ch != "*")
66     total = [0] * (n + m - 1)
67     for sigma in alphabet:
68         s_ind = [1 if ch == sigma else 0 for ch in S]
69         p_ind = [1 if ch == sigma else 0 for ch in reversed(P)] # wildcard->0 always
70         conv = multiply(s_ind, p_ind)
71         for i, v in enumerate(conv):
72             total[i] += v
73     out = []
74     for j in range(n - m + 1):
75         if total[j + m - 1] == fixed:
76             out.append(j)
77     return out
78
79
80 def match_naive(S: str, P: str) -> List[int]:
81     """O(nm) reference matcher supporting '*' wildcard in P."""
82     n, m = len(S), len(P)
83     out = []
84     for j in range(n - m + 1):
85         if all(P[k] == "*" or P[k] == S[j + k] for k in range(m)):

```

```
86         out.append(j)
87     return out
88
```